

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

Proactive Intervention Desk

Architecture Design for an Ambient Multi-Agent System

Divyani Saxena

September 2026

1 Introduction

A Multi-Agent System (MAS) in which all participating agents operate ambiently is proposed. The design, coordination patterns, and memory models of this MAS architecture were derived from the following reference literature and resources:

- **Multi-Agent Systems (MAS):** General MAS coordination vocabulary covering swarm, sequential handoff, and debate topologies as distinct architecture patterns rather than a single default pattern.
- **Ambient Agent Frameworks:** Conceptual framing for background, non-intrusive autonomous monitoring agents.
- **How Salesforce Manages Solutions:** Operational strategies for enterprise agent orchestration.
- **Memory Types in Agentic AI:** Working vs. episodic vs. semantic memory as the standard three-way split for agentic architectures. Confirms the three-tier memory design in `memory/` rather than a single flat store.
- **Memory in Multi-Agent Systems:** Blackboard-style shared memory where agents write structured facts to a common board rather than passing full context peer-to-peer, avoiding silent fact disagreement.
- **Connect Everything with Agentforce MuleSoft:** Agent-to-Agent (A2A) protocol enabling independently-built agents to exchange structured messages without direct database sharing.
- **What is Salesforce:** Architectural context for CRM-integrated autonomous workflows.
- **Model-Agnostic Hooks:** Explainability hooks functioning regardless of underlying models, attaching reasoning and citations as structured metadata alongside output.
- **Multi-Agent Systems for Ambient Intelligent Applications:** Evaluation framework focusing on natural distribution, agent intelligence, multi-agent planning, and context-awareness. Justifies treating the Life-Event correlation layer as multi-agent planning over state boards.
- **Non-Negotiables & OpenTelemetry:** Operational requirements including OpenTelemetry usage for tracing distributed agent executions.
- **Dynamic PII Masking Systems:** Interface-layer data protection techniques for dynamic agent inputs.

- **K2view PII Masking**: Data privacy compliance and masking patterns.
- **Blackboard Pattern with MCPs**: Canonical three-part blackboard architecture: Shared Board, Knowledge Sources (Agents), and a Control Component.
- **RAG Freshness**: Architectural patterns for real-time web retrieval and state invalidation.

2 Agent Memory Taxonomy

The system utilizes a three-tier memory topology:

Working Memory: Per-customer, session-scoped, and discarded once the active analysis session is completed.

Episodic Memory: Per-customer persistent memory storing interventions, outcomes, and current life-phase state. Persists across multiple weeks and is readable by downstream agents (Offer, Retention, Synthesis) without re-derivation.

Semantic Memory: System-wide memory shared across customers, containing eligibility rules and compliance policies. Managed via A2A protocols.

3 Multi-Agent System Architecture Design

3.1 Ingestion via Swarm Topology

The **Usage**, **Transaction**, and **Support** agents form an ingestion swarm. They operate concurrently and independently over customer data streams without blocking each other. Each publishes structured conclusions (`key: value, confidence`) directly to a per-customer *Shared State Board*. Raw reasoning traces are omitted to allow clean context integration for downstream agents.

3.2 Correlation & Event Inference

The **Life-Event Inference Agent** operates on an agent-dependent trigger model. Rather than firing on isolated raw signals, it evaluates state changes when the *StateBoard* accumulates correlated findings from ≥ 2 independent swarm agents within a 21-day recency window.

3.3 Conflict Resolution & Synthesis

An **Agent Debate** mechanism handles candidate selection inside the **Synthesis Agent** via explicit priority rules (`agents/synthesis_agent.py::STATE.PRIORITY`) when multiple life-event candidates conflict. Multi-pass round-robin text drafting was scoped down to a single templated generation step to minimize latency.

3.4 Memory & Decay Mechanics

- **Working Memory**: Implemented via the *StateBoard* (`memory/state.board.py`). Decays naturally as new findings supersede outdated entries.
- **Episodic Memory**: Backed by customer JSON records (`memory/episodic.memory.py`). Tracks active `life_phase` and intervention history, utilizing a 90-day exponential decay half-life.
- **Semantic Memory**: Managed in a centralized policy file accessed by Offer and Critique agents.

3.5 Safety, Compliance, and Observability

- **Critique-Refiner:** Acts as a mandatory LLM validation gate evaluating cost feasibility and compliance prior to Human-in-the-Loop (HITL) review.
- **Deterministic Guardrail:** Runs parallel checks for legal, fraud, or safety keywords to immediately bypass normal agent pipelines and route cases directly to specialist human queues.
- **OpenTelemetry Tracing:** Utilizes `opentelemetry-sdk` (`TracerProvider/Span`) for real parent-child trace mapping, exported directly to local JSONL logs.
- **Data Privacy:** Interface layer executes dynamic PII masking using HMAC-SHA256 tokenization.
- **Orchestrator Control:** `main.py::Orchestrator` functions as the Control Component of the Blackboard design, dynamically scheduling agent execution based on state changes.

4 Agent and Trigger Classifications

Agent	Trigger Type	Trigger Description
Usage	Event + Time-based	Direct usage events; nightly aggregated rollups
Support	Event-based	Creation of new tickets or inbound messages
Transaction	Event-based	Real-time financial transaction streams
KYC	Event + Time-based	KYC attribute updates; periodic re-verification schedules
Guardrail	Independent Event	Legal/fraud/safety keyword match (immediate bypass)
Life-Event Inference	Agent-dependent	Correlated multi-source signals on StateBoard
Synthesis	Agent-dependent	Ingestion completion by swarm agents
Offer / Eligibility	Agent-dependent	Life-event opportunity produced by Synthesis
Retention / Action	Agent-dependent	Consensus between Synthesis and Offer
Critique-Refiner	Agent-dependent	Direct output generation from Action agent

Table 1: Agent classification and trigger mechanics

5 System Architecture Flowchart

The structural and dataflow topology of the ambient multi-agent system is illustrated in Figure 1.

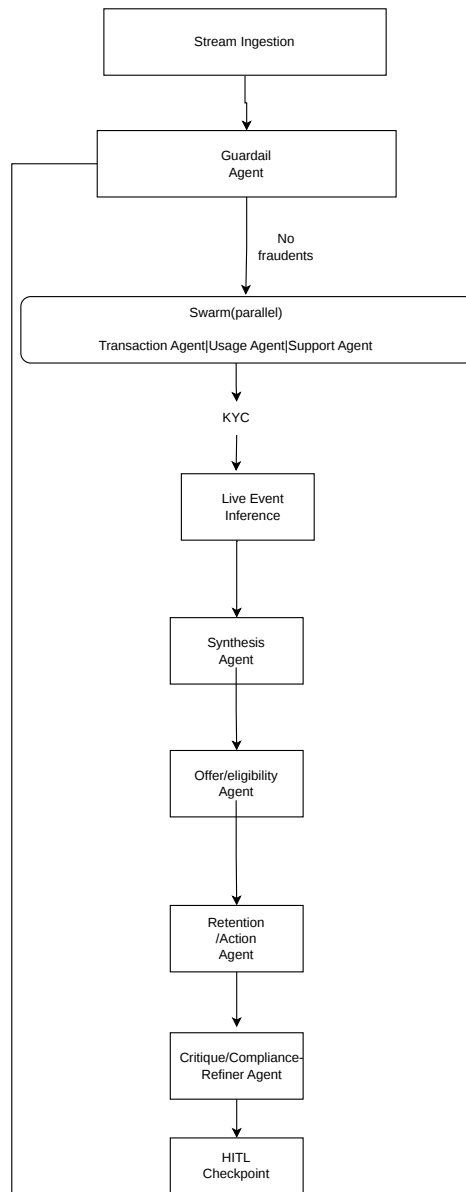


Figure 1: Proposed Ambient Multi-Agent System Architecture